

Estudo: Grid, @media, Flexbox e Menu

Site Patas Unidas · resumo para estudar com calma depois da entrega. Tudo aqui está no seu `estilo.css`.

1. O Grid de 12 colunas

```
main {  
  display: grid; /* liga o grid no main */  
  grid-template-columns: repeat(12, 1fr); /* cria 12 colunas iguais */  
  gap: var(--espaco-medio); /* espaço entre as caixas (16px) */  
}
```

- `display: grid`: transforma o `main` numa "grade".
- `repeat(12, 1fr)`: repete 12 vezes uma coluna de `1fr`.
- `fr` = "fração": divide o espaço em partes iguais. 12 colunas de `1fr` = cada uma ocupa 1/12 da largura.
- `gap`: o espaço entre as colunas e as linhas.

A barra de chocolate

O grid é uma barra de chocolate com **12 quadradinhos**. O `span` diz quantos quadradinhos cada `section` come. Cada cor abaixo é uma section.

span 12		1 por linha ($12 \div 1 = 12$)
span 6		2 por linha ($12 \div 2 = 6$)
span 4		3 por linha ($12 \div 3 = 4$)
span 3		4 por linha ($12 \div 4 = 3$)

Regra: quantas sections você quer por linha? Faça $12 \div$ **esse número**. O resultado é o span.

2. O que é @media

`@media` é um "SE" do CSS: **se a tela for de tal tamanho, aplique estas regras**.

```
@media(min-width:768px) { /* SE a tela tiver no MÍNIMO 768px... */  
  section { /* ...então, na section... */  
    grid-column: span 6; /* ...ocupe 6 colunas */  
  }  
}
```

- `min-width: 768px` = "de 768px **para cima**".
- Dentro do `@media` vai um bloco CSS normal: seletor + `{` + propriedades + `}`.
- São **2 pares de chaves**: um do `@media` e um do seletor.

3. Os seus 5 breakpoints

Breakpoint	Tipo de tela	O que muda
(sem @media)	celular	<code>section</code> ocupa 12 colunas (1 por linha)
576px	celular grande	<code>body</code> com padding maior (--espaco-muito-grande)
768px	tablet	<code>section</code> span 6 (2 por linha)
992px	notebook	<code>section</code> span 4 (3 por linha)
1200px	desktop	<code>section</code> span 3 (4 por linha)
1400px	tela muito grande	<code>main</code> com <code>max-width: 1320px</code> e <code>margin: 0 auto</code> (centralizado)

max-width e margin: 0 auto

- `max-width: 1320px`: o `main` cresce, mas **nunca passa** de 1320px.
- `margin: 0 auto`: `0` em cima e embaixo; `auto` divide o espaço que sobra entre esquerda e direita, **centralizando**.

4. Estratégia: Mobile First

Você escreveu o CSS **primeiro para o celular** (sem @media) e depois foi **adicionando** mudanças para telas maiores com `min-width`. Isso se chama **Mobile First** ("celular primeiro").

5. Quem vem por último ganha

Quando duas regras valem ao mesmo tempo para o mesmo elemento, **vence a que está mais embaixo no arquivo**.

Exemplo do erro que aconteceu: um bloco de 768px (span 6) colado **depois** do 992px (span 4). Numa tela de 1000px os dois valem, e o de baixo (span 6) apagava o span 4.

Com `min-width`, coloque os @media do **menor para o maior**.

6. Erros que você cometeu (e como evitar)

Erro	Certo	Por quê
<code>var(--espaco-muito-grande)</code>	<code>var(--espaco-muito-grande)</code>	O nome da variável tem que ser idêntico ao do <code>:root</code> . Dica: copie e cole o nome.
<code>max-width: 1320;</code>	<code>max-width: 1320px;</code>	Tamanho sem unidade (<code>px</code>) é ignorado pelo navegador.
<code>928px</code>	<code>992px</code>	Número digitado trocado. Confira sempre.
Bloco colado 2 vezes	Colar só uma vez	O Ctrl+V cola o que ficou copiado da última vez.
Esquecer a <code>{</code> depois do seletor	<code>section {</code>	Sem ela o bloco quebra.
Não salvar	Ctrl+S	Sem salvar, o navegador não vê a mudança.

7. Para treinar depois

A pasta `meu projeto 1\treino-media\` tem 5 exercícios de @media em `treino.css`. Os exercícios 2, 3 e 4 ainda precisam ser refeitos **com** @media.

8. Flexbox: alinhar coisas DENTRO das caixas

O **Grid** organiza as áreas grandes da página (as sections). O **Flexbox** organiza o que fica **dentro** de cada caixa: título, texto, botão, campos do formulário.

```
section {  
  display: flex;           /* liga o flexbox: a section vira um "contentor" */  
  flex-direction: column; /* itens um embaixo do outro (row = lado a lado) */  
  align-items: flex-start; /* alinha os itens no começo (esquerda) */  
  gap: var(--espaco-pequeno); /* espaço entre os itens */  
}
```

Propriedade	Para que serve
<code>display: flex</code>	Liga o Flexbox. O elemento vira o "contentor" e os filhos viram "itens".
<code>flex-direction</code>	<code>row</code> = lado a lado (padrão). <code>column</code> = um embaixo do outro.
<code>justify-content</code>	Distribui os itens na direção principal. <code>space-between</code> = um em cada ponta.
<code>align-items</code>	Alinha os itens na outra direção. <code>center</code> = no meio; <code>flex-start</code> = no começo.
<code>flex-wrap: wrap</code>	Se não couber na linha, o item desce para a linha de baixo.
<code>gap</code>	Espaço entre os itens.
<code>align-self</code>	Alinha só um item, diferente dos outros.

Onde você usou

Componente	Propriedades
Cartões (<code>section</code>)	<code>flex</code> , <code>column</code> , <code>align-items: flex-start</code> , <code>gap</code>
Contato (<code>address</code>)	<code>flex</code> , <code>column</code> , <code>gap</code>
Formulário (<code>form</code>)	<code>flex</code> , <code>column</code> , <code>gap</code> ; no botão: <code>align-self: flex-start</code>
Grupos (<code>fieldset</code>)	<code>flex</code> , <code>flex-wrap: wrap</code> , <code>align-items: center</code> , <code>gap</code>

9. Menu responsivo: dropdown e hambúrguer

A estrutura no HTML

```
<header class="cabecalho">
  <a class="logo">Patas Unidas</a>
  <input type="checkbox" id="menu-toggle" class="menu-toggle">  <!-- guarda aberto/fechado -->
  <label for="menu-toggle" class="hamburguer">&#9776;</label>  <!-- o ícone ≡ -->
  <nav class="menu">
    <ul class="menu-lista">
      <li>Sobre</li>
      <li class="tem-submenu">Como ajudar
        <ul class="submenu"> ...3 links... </ul>  <!-- o dropdown -->
      </li>
      <li>Contato</li>
    </ul>
  </nav>
</header>
```

Truque 1: o hambúrguer sem JavaScript (checkbox)

- O **checkbox** fica escondido (`display: none`), mas ele continua funcionando.
- O **label** com `for="menu-toggle"` é ligado ao checkbox: clicar no ≡ marca/desmarca o checkbox.
- No celular o menu começa fechado: `max-height: 0; overflow: hidden;`
- Quando o checkbox está marcado, este seletor abre o menu:

```
.menu-toggle:checked ~ .menu {  /* checkbox MARCADO + "~" = o .menu que vem depois dele */
  max-height: 500px;
}
```

- `:checked` é uma **pseudo-classe**: vale só quando o checkbox está marcado.
- `transition: max-height 0.3s ease` faz o menu abrir deslizando.

Truque 2: o dropdown (submenu)

- O item pai recebe `position: relative`, e o submenu recebe `position: absolute; top: 100%;`. Assim o submenu "flutua" logo abaixo do pai.
- Escondido por padrão: `opacity: 0; visibility: hidden;`
- Aparece com as pseudo-classes `:hover` (mouse em cima) e `:focus-within` (navegando pelo teclado):

```
.tem-submenu:hover .submenu,
.tem-submenu:focus-within .submenu {
  opacity: 1;
  visibility: visible;
}
```

O @media que troca tudo

	Celular (padrão)	A partir de 768px
Ícone ☰	aparece	display: none
Menu	fechado, abre no clique	sempre visível
Lista	column (vertical)	row (horizontal)
Submenu	sempre visível, com recuo	escondido, aparece no hover

10. Estados dos botões, cartões e validação

Pseudo-classes são "momentos" de um elemento. O CSS dentro delas só vale naquele momento.

Pseudo-classe	Quando acontece	O que você fez
:hover	mouse em cima	cor muda, botão sobe (translateY(-2px)), sombra maior
:focus-visible	chegou no elemento pelo Tab do teclado	contorno (outline) visível
:active	no momento do clique	botão "afunda" (translateY(2px))
:disabled	botão desativado	opacity: 0.5, cursor: not-allowed
:focus	campo do formulário selecionado	borda azul + brilho (box-shadow)
:user-valid	campo preenchido certo (depois que a pessoa mexeu)	borda --sucesso (verde)
:user-invalid	campo preenchido errado (depois que a pessoa mexeu)	borda --erro (vermelho)

transition: a mudança suave

```
transition: background-color 0.3s ease;
/*      O QUE muda      QUANTO tempo  COMO (ease = começa e termina devagar) */
```

Sem transition, a mudança acontece de uma vez. Com ela, a cor ou a posição muda aos poucos.

Como o navegador sabe se o campo está certo?

Pelos atributos do HTML: required (obrigatório), type="email" (tem que ter @) e pattern (formato, ex.: CPF 000.000.000-00). O CSS só pinta de verde ou vermelho.

11. Componentes de feedback: badge, alerta e toast

Feedback = o site "responde" ao usuário. São 3 tipos:

Componente	O que é	Truque principal no CSS
Badge (etiqueta)	Pílula pequena colorida que classifica algo: "Disponível", "Adotado", "Cão"	<code>display: inline-block</code> , fonte pequena, <code>border-radius: 999px</code> , uma cor por classe
Alerta	Caixa de mensagem: sucesso, atenção, erro, informação	<code>border-left: 6px solid</code> + fundo claro da mesma cor
Toast	Notificação que aparece no canto da tela, por cima de tudo	<code>position: fixed</code> + <code>right / bottom</code> + <code>@keyframes</code> para entrar

Uma classe base + uma classe de cor

```
<span class="badge badge-sucesso">Disponível</span>
/*  .badge          = formato (igual para todas)
    .badge-sucesso = só a cor (muda em cada uma) */
```

Assim você escreve o formato uma vez só e troca apenas a cor. As cores vêm das variáveis: `--sucesso`, `--aviso`, `--erro`, `--info`.

@keyframes: animação

```
@keyframes toast-entrar {
  from { opacity: 0; transform: translateY(20px); } /* começa invisível e mais embaixo */
  to   { opacity: 1; transform: translateY(0); }   /* termina visível no lugar */
}
.toast { animation: toast-entrar 0.5s ease 0.5s both; }
/*          nome          duração  espera */
```

role="status" e role="alert"

Atributos do HTML para acessibilidade: avisam o leitor de tela (usado por pessoas cegas) que ali tem uma mensagem. `alert` é para erro (urgente), `status` para o resto.

12. Biblioteca externa: Day.js (datas)

Biblioteca = código pronto que outra pessoa fez. É como uma **caixa de ferramentas emprestada**: em vez de fazer a furadeira do zero, você pega uma pronta e só aprende a ligar.

CDN = um site que guarda a biblioteca para você usar pelo link, sem baixar nada.

Passo 1: trazer a biblioteca (no HTML)

```
<script src="https://cdn.jsdelivr.net/npm/dayjs@1/dayjs.min.js"></script>
<script src="../js/pets.js"></script>
/* A biblioteca vem ANTES do pets.js, porque o pets.js usa ela.
   Se ficar depois: erro "dayjs is not defined" (dayjs não existe) */
```

Passo 2: usar (no pets.js)

Código	O que faz	Resultado
dayjs()	pega a data de hoje	30/09/2026
dayjs().diff('2026-08-10', 'day')	conta quantos dias se passaram desde aquela data até hoje	51
dayjs('2026-08-10').format('DD/MM/YYYY')	escreve a data no jeito brasileiro (Dia/Mês/Ano)	10/08/2026

No site, cada pet ganhou o campo **chegada** e a lista mostra: *"Chegou em 10/08/2026 (há 51 dias no abrigo)"*.

13. JavaScript separado em vários arquivos

A ideia é a mesma de uma **cozinha de restaurante**: um cozinheiro faz a salada, outro a carne, outro a sobremesa. Cada um cuida de **uma coisa só**. Se a sobremesa der errado, você sabe exatamente onde procurar.

Arquivo	Cuida só de...	Usa função de...
armazenamento.js	salvar e carregar favoritos (localStorage)	ninguém
pets.js	mostrar os pets na tela e o clique de favoritar	armazenamento.js
spa.js	trocar de página (início, adoção, contato)	pets.js
validacao.js	regras do formulário (RegExp) e <code>validarCampo()</code>	ninguém
cadastro.js	eventos do formulário (blur, input, submit)	validacao.js
main.js	toast e menu no celular	ninguém

A regra de ouro: a ORDEM dos scripts

```
<script src="../../js/armazenamento.js"></script> /* 1º: quem FORNECE a função */
<script src="../../js/pets.js"></script>           /* 2º: usa o armazenamento */
<script src="../../js/spa.js"></script>             /* 3º: usa o pets */
<script src="../../js/main.js"></script>
```

O navegador lê de cima para baixo. Se o `pets.js` viesse antes do `armazenamento.js`, ele tentaria usar `salvarFavoritos()` antes dela existir e daria erro.

Cuidados

1. Não repetir o mesmo nome de variável ou função em arquivos diferentes (senão um "atropela" o outro).
2. A comunicação vai para um lado só: spa → pets → armazenamento. Nunca o contrário.

Para saber depois: import e export

Existe um jeito mais avançado (ES6 Modules), em que o arquivo escreve `export` na função e o outro escreve `import` para pegá-la. Aí a ordem não importa mais, mas o site só funciona aberto pelo Live Server. Vamos estudar isso com calma depois.

14. Testar e caçar bugs (DevTools)

Bug = o site faz algo diferente do que deveria. Testar é **tentar quebrar o site de propósito**: digitar coisa errada, tirar a internet, clicar fora de ordem.

Aperte **F12** no navegador para abrir o DevTools:

Aba	Para que serve
Console	mostra os erros em vermelho, com o nome do arquivo e a linha. Também dá para testar código ali.
Elements	mostra o HTML "ao vivo" (atributos como <code>required</code> e <code>hidden</code>)
Network (Rede)	mostra os arquivos baixados. Tem a opção Offline para simular sem internet.

Os 4 bugs que achamos

Bug	Causa	Correção
Sem internet, a lista de pets ficava vazia	Day.js vem da CDN; sem internet, <code>dayjs</code> não existe → erro	<code>if (typeof dayjs !== 'undefined') no pets.js</code>
Botão ativo com nome e e-mail vazios	<code>checkValidity()</code> só olha campos com <code>required</code>	pus <code>required</code> no nome e no email
"Maria-Clara" dava nome inválido	a RegEx não aceitava <code>-</code> nem <code>'</code>	<code>/^[A-Za-zÀ-ÿ ']{3,}\$/</code>
Mensagem de sucesso não sumia	ninguém escondia ela de novo	<code>mensagem.hidden = true</code> no evento input

Anexo: textos entregues na atividade

Texto (a): configurações do grid e os 5 breakpoints

O layout do site Patas Unidas foi feito com CSS Grid. O grid foi aplicado no elemento main com display: grid, e ele foi dividido em 12 colunas de larguras iguais com grid-template-columns: repeat(12, 1fr). A unidade fr divide o espaço disponível em frações iguais, então cada coluna ocupa 1/12 da largura. O espaço entre as colunas e as linhas é definido pelo gap, que usa a variável --espaco-medio (16px) do Design System, mantendo o espaçamento consistente com o resto do site.

A quantidade de colunas que cada section ocupa é definida com grid-column: span. Sem nenhum @media, a section usa span 12, ou seja, ocupa a linha inteira, e as seções ficam uma embaixo da outra no celular. Conforme a tela aumenta, os @media mudam quantas colunas cada section ocupa, sempre com números que dividem 12 sem sobrar espaço (12, 6, 4 e 3).

Os 5 breakpoints são:

1. 576px (celulares grandes): o padding do body aumenta de --espaco-grande (24px) para --espaco-muito-grande (32px), dando mais respiro nas laterais.
2. 768px (tablets): cada section passa a ocupar 6 colunas, ficando 2 por linha.
3. 992px (notebooks): cada section ocupa 4 colunas, ficando 3 por linha.
4. 1200px (desktops): cada section ocupa 3 colunas, ficando 4 por linha.
5. 1400px (telas muito grandes): o main recebe max-width: 1320px e margin: 0 auto, para o conteúdo não esticar demais e ficar centralizado na tela.

Texto (b): breakpoints em px e a estratégia

Breakpoints usados: 576px, 768px, 992px, 1200px e 1400px. A estratégia usada é Mobile First: o CSS base foi escrito para o celular (section com span 12, uma embaixo da outra) e os @media usam min-width para acrescentar mudanças só quando a tela fica maior. Assim, as telas maiores vão ganhando mais colunas: 2 por linha no tablet, 3 no notebook e 4 no desktop. No 576px aumenta o espaçamento lateral do body e no 1400px o main é limitado a 1320px e centralizado, para o conteúdo não ficar esticado em telas muito largas. Os valores seguem os tamanhos mais comuns de celular, tablet, notebook e monitor.

Flexbox: módulos estruturais

Nome	Propriedades
Cartões de conteúdo (section)	display: flex; flex-direction: column; align-items: flex-start; gap: var(--espaco-pequeno). Organiza título, texto e botão de cada cartão um embaixo do outro, alinhados à esquerda, com espaçamento do Design System.
Bloco de contato (address)	display: flex; flex-direction: column; gap: var(--espaco-minimo). Empilha telefone, e-mail e endereço em coluna com espaçamento uniforme entre as linhas.
Formulário de cadastro (form)	display: flex; flex-direction: column; gap: var(--espaco-medio). No botão de enviar: align-self: flex-start. Organiza os grupos de campos em coluna e mantém o botão alinhado no início, sem esticar.
Grupos de campos do formulário (fieldset)	display: flex; flex-wrap: wrap; align-items: center; gap: var(--espaco-pequeno). Coloca rótulo e campo lado a lado, centralizados na vertical, e com flex-wrap os campos descem para a linha de baixo quando não cabem na tela.

Menu responsivo (dropdown e hambúrguer)

Para criar o menu de navegação do site Patas Unidas, adicionei no HTML um header com a classe .cabecalho contendo o logo, um checkbox escondido (#menu-toggle), um label com o ícone de hambúrguer (.hamburger) e o nav (.menu) com uma lista (.menu-lista). O item "Como ajudar" (.tem-submenu) tem uma lista interna (.submenu), que é o dropdown com os links Seja voluntário, Doações e Adote um pet.

O CSS foi feito com a estratégia Mobile First. O .cabecalho usa display: flex, justify-content: space-between e flex-wrap: wrap, deixando o logo à esquerda e o ícone à direita. No celular, o menu fica fechado com max-height: 0 e overflow: hidden, e a lista usa flex-direction: column para os links ficarem um embaixo do outro. O checkbox fica escondido com display: none, e o label funciona como botão do hambúrguer: quando ele é clicado, o checkbox fica marcado e o seletor .menu-toggle:checked ~ .menu muda o max-height para 500px, abrindo o menu. A transition: max-height 0.3s ease faz o menu abrir deslizando.

No @media (min-width: 768px), a partir do tablet, o ícone de hambúrguer some com display: none, o .menu volta a aparecer com max-height: none e overflow: visible, e a .menu-lista muda para flex-direction: row, virando uma navegação horizontal.

O submenu fica escondido por padrão no desktop com position: absolute, opacity: 0 e visibility: hidden. O item pai (.tem-submenu) recebe position: relative para o submenu aparecer logo abaixo dele (top: 100%). As pseudo-classes :hover e :focus-within mostram o dropdown (opacity: 1 e visibility: visible) quando o mouse passa em cima ou quando o usuário navega pelo teclado. A transition em opacity e transform faz o submenu aparecer suavemente, descendo alguns pixels com transform: translateY. No hover dos links, a pseudo-classe :hover troca a cor de fundo pela variável --cor-secundaria do Design System, e todas as cores e espaçamentos usam as variáveis do :root.

Estilização de cartões e formulários

Para os estados interativos do site Patas Unidas, usei as variáveis do Design System e pseudo-classes do CSS, sempre com transições para as mudanças ficarem suaves.

Botões (.botao e button): o estado normal tem fundo `--cor-secundaria`, texto `--cor-botao`, bordas arredondadas (`border-radius: 999px`), uma sombra embaixo (`box-shadow`) e `transition: background-color 0.3s ease, transform 0.2s ease, box-shadow 0.2s ease`.

- `:hover` (mouse em cima): o fundo muda para `--cor-hover`, o botão sobe 2px com `transform: translateY(-2px)` e a sombra fica maior, mostrando que ele é clicável.

- `:focus-visible` (navegação pelo teclado com Tab): aparece um contorno com `outline: 3px solid --cor-botao` e `outline-offset: 3px`, garantindo acessibilidade para quem não usa mouse.

- `:active` (momento do clique): o botão desce com `transform: translateY(2px)` e a sombra diminui, dando a sensação de que foi pressionado.

- `:disabled` (botão desativado): `opacity: 0.5`, `cursor: not-allowed` e sem sombra nem movimento, deixando claro que não dá para clicar.

Cartões (section): fundo `--cor-formularios` (branco), `padding --espaco-grande`, `border-radius: 16px` e `box-shadow` leve. No `:hover` o cartão sobe 4px com `transform: translateY(-4px)` e a sombra aumenta, com `transition` de 0.3s em `transform` e `box-shadow`.

Formulário: o form também virou um cartão branco com sombra e bordas arredondadas. Os fieldset têm borda de 2px e `border-radius: 12px`, e a legend fica em negrito com a `--cor-botao`. Os campos (input e select) têm `padding`, `border: 2px solid cinza`, `border-radius: 8px` e `transition` em `border-color` e `box-shadow`.

- `:focus` (campo selecionado): a borda muda para `--cor-botao` e aparece um brilho em volta com `box-shadow`, e o `outline` padrão foi removido.

- Validação visual: usei as pseudo-classes `:user-valid` e `:user-invalid`, que só aparecem depois que o usuário mexe no campo, para não mostrar erro antes da hora. Elas funcionam junto com os atributos do HTML (`required`, `type="email"` e `pattern` nos campos de telefone, CPF e CEP). Quando o campo está correto (`:user-valid`), a borda fica com a variável `--sucesso` (verde) e o fundo verde claro. Quando está errado (`:user-invalid`), a borda fica com a variável `--erro` (vermelho) e o fundo vermelho claro. Assim o usuário entende na hora se preencheu certo, sem precisar ler nenhuma mensagem.

Biblioteca externa (Day.js)

No site do Instituto Patas Unidas eu usei a biblioteca Day.js, que serve para trabalhar com datas em JavaScript. Escolhi ela porque é pequena, fácil de usar e resolve um problema que seria mais trabalhoso de fazer só com JS puro: calcular quantos dias se passaram entre duas datas e mostrar a data no formato brasileiro.

O propósito no projeto é mostrar, na página de adoção, há quanto tempo cada pet está no abrigo. Assim o visitante vê, por exemplo: "Chegou em 10/08/2026 (há 51 dias no abrigo)". Isso ajuda a chamar atenção para os pets que estão esperando há mais tempo.

Passos que eu segui:

1. Importei a biblioteca pela CDN, colocando no index.html a linha: `<script src="https://cdn.jsdelivr.net/npm/dayjs@1/dayjs.min.js"></script>`

2. Coloquei esse script ANTES do meu pets.js, porque o pets.js usa a biblioteca. Se ficasse depois, daria erro de "dayjs is not defined".

3. No array de pets, adicionei um campo novo chamado chegada, com a data no formato ano-mês-dia (ex: '2026-08-10').

4. Dentro da função renderizarPets, usei dois métodos básicos da biblioteca: `dayjs().diff(pet.chegada, 'day')` calcula quantos dias se passaram desde a chegada até hoje;

`dayjs(pet.chegada).format('DD/MM/YYYY')` mostra a data no jeito brasileiro.

5. Coloquei o resultado dentro do `<li>` de cada pet com template literal.

Sobre conflitos: a Day.js cria só uma função global chamada `dayjs`, e no meu código não tem nenhuma variável com esse nome, então não atrapalha o resto do site. Os meus scripts continuam separados (`pets.js`, `spa.js`, `main.js`) e a biblioteca carrega primeiro, então não tem problema de ordem de carregamento.

JavaScript modular (arquivos separados)

No meu projeto eu não usei `import` e `export`. Eu fiz a separação em vários arquivos JavaScript menores, e cada um cuida de uma coisa só. Assim o código fica mais organizado e fácil de achar.

O critério que usei foi: o arquivo que mexe na tela não mexe no `localStorage`, e o arquivo que valida o formulário não cuida do envio.

Como ficou a pasta `js`:

- `armazenamento.js`: só salva e carrega os pets favoritos no `localStorage`. Tem as funções `carregarFavoritos()` e `salvarFavoritos()`.
- `pets.js`: só mostra a lista de pets na tela e cuida do clique para favoritar. Quando precisa salvar, ele chama `salvarFavoritos()`.
- `spa.js`: só troca as páginas do site (início, adoção e contato). Quando abre a página de adoção, ele chama `renderizarPets()`, que está no `pets.js`.
- `validacao.js`: só tem as regras do formulário (RegEx e mensagens de erro) e a função `validarCampo()`.
- `cadastro.js`: só cuida dos eventos do formulário (quando sai do campo, quando digita e quando envia). Ele chama `validarCampo()` do `validacao.js`.
- `main.js`: cuida do toast e do menu no celular.

Como eles se comunicam: um arquivo usa a função do outro chamando pelo nome. Por isso a ordem dos scripts no HTML é importante: o arquivo que tem a função precisa vir antes do arquivo que usa ela. No `index.html` a ordem é `armazenamento.js`, `pets.js`, `spa.js` e `main.js`. No `cadastro.html` o `validacao.js` vem antes do `cadastro.js`.

Para não dar problema, tomei cuidado para não repetir nome de variável ou de função em arquivos diferentes. Também fiz a comunicação ir para um lado só: o `spa.js` usa o `pets.js`, e o `pets.js` usa o `armazenamento.js`, mas nunca o contrário.

Com essa separação, se eu precisar mudar o jeito de salvar os dados, eu mexo só no `armazenamento.js` e o resto do site continua funcionando.